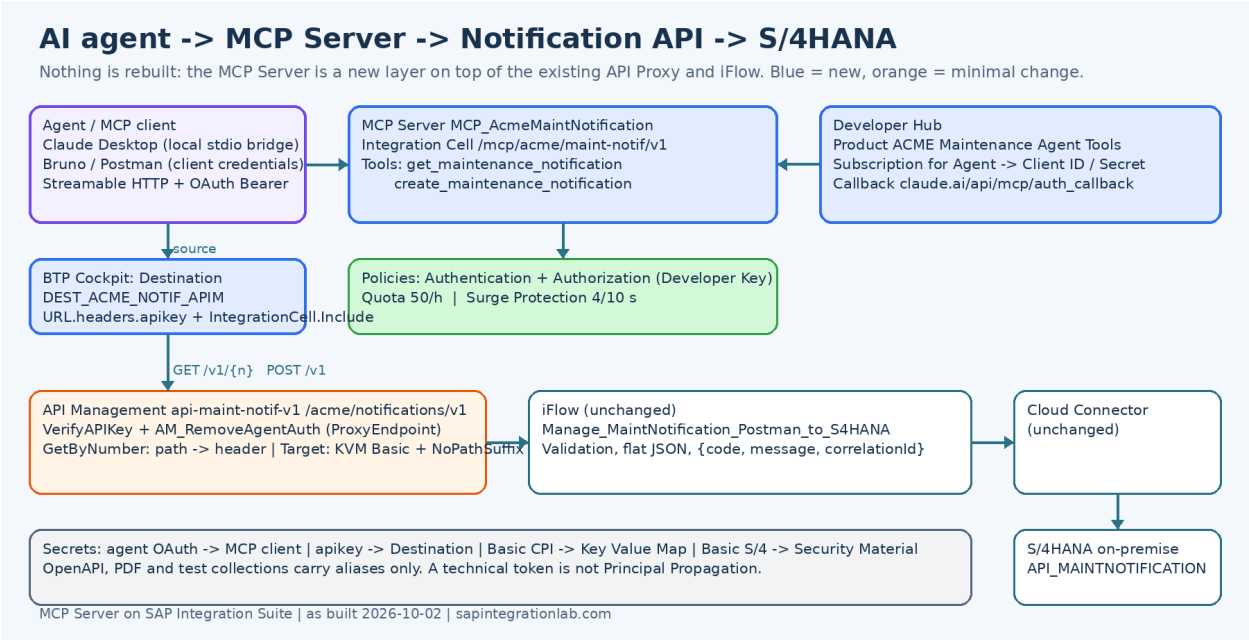
Exposing SAP S/4HANA Plant Maintenance to an AI Agent with an MCP Server on SAP Integration Suite

Step-by-step manual · SAP Integration Suite (Integration Cell, API Management, Developer Hub) · S/4HANA on-premise via Cloud Connector · Claude Desktop as the agent · October 2026 · Gabriel Luces · sapintegrationlab.com

1. What you will build

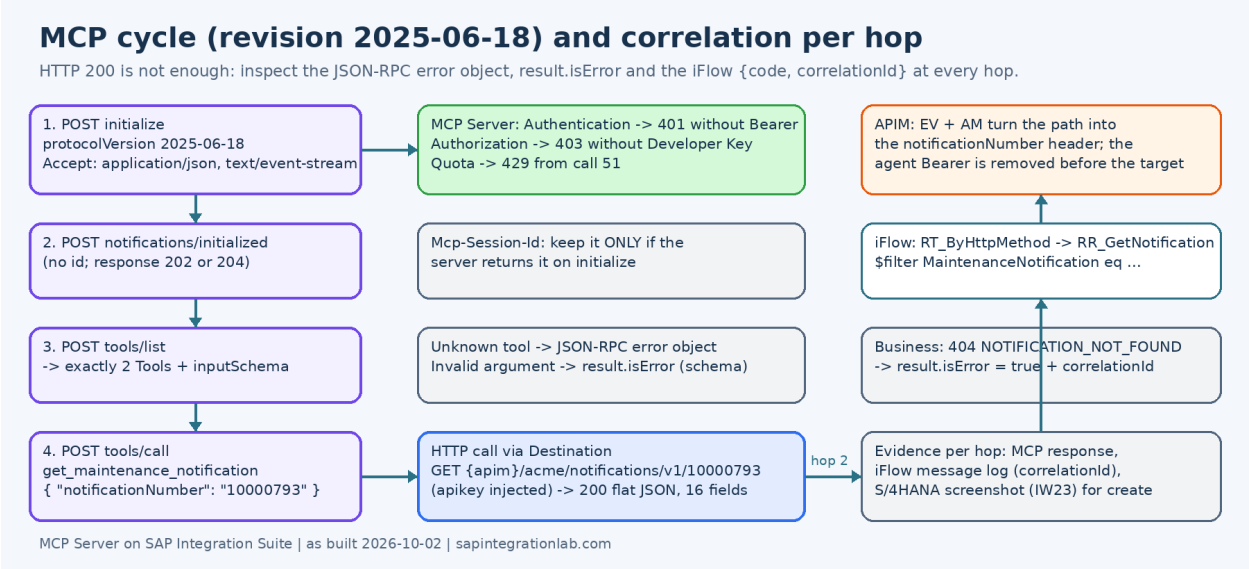
By the end of this manual an AI agent (Claude Desktop) can ask "what is the status of maintenance notification 10000793?" or "create a malfunction notification for equipment 10000" and the request lands in a real SAP S/4HANA system through SAP Integration Suite. You will **not** write a new integration flow: an existing REST API is wrapped as an **MCP Server** with two Tools.

Tool	Operation	Backend path	Safety profile
get_maintenance_notification	GET /v1/{notificationNumber}	Read notification (QMNUM)	Read-only, idempotent
create_maintenance_notification	POST /v1	Create M2 notification	Writes real data, not idempotent, requires user confirmation



End-to-end architecture: Agent → MCP Server (Integration Cell) → BTP Destination → API Proxy → existing iFlow → Cloud Connector → S/4HANA

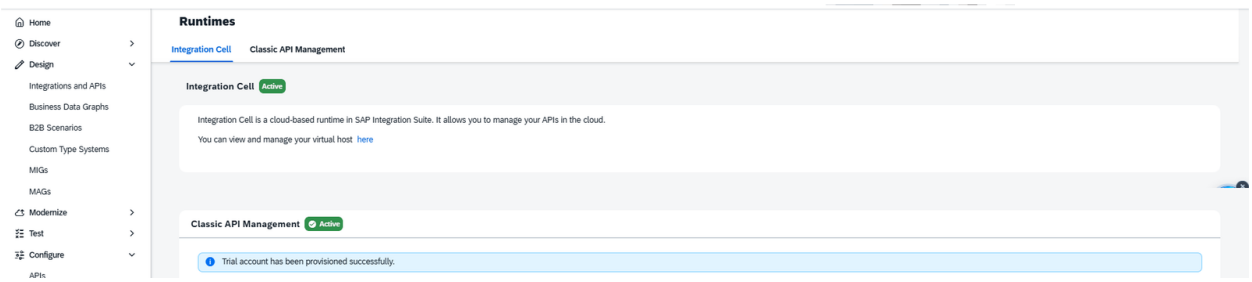
Flow of one call: the agent sends JSON-RPC (tools/call) with an OAuth Bearer token to the MCP Server running on Integration Cell. The MCP Server resolves the Tool to an HTTP operation of the OpenAPI document, calls the BTP Destination (which injects the API key), API Management validates the key, translates the path parameter into the header the iFlow expects, authenticates against Cloud Integration with Basic Auth from a Key Value Map, the iFlow calls S/4HANA API_MAINTNOTIFICATION through the Cloud Connector and returns a flat JSON that the MCP Server hands back as structuredContent.



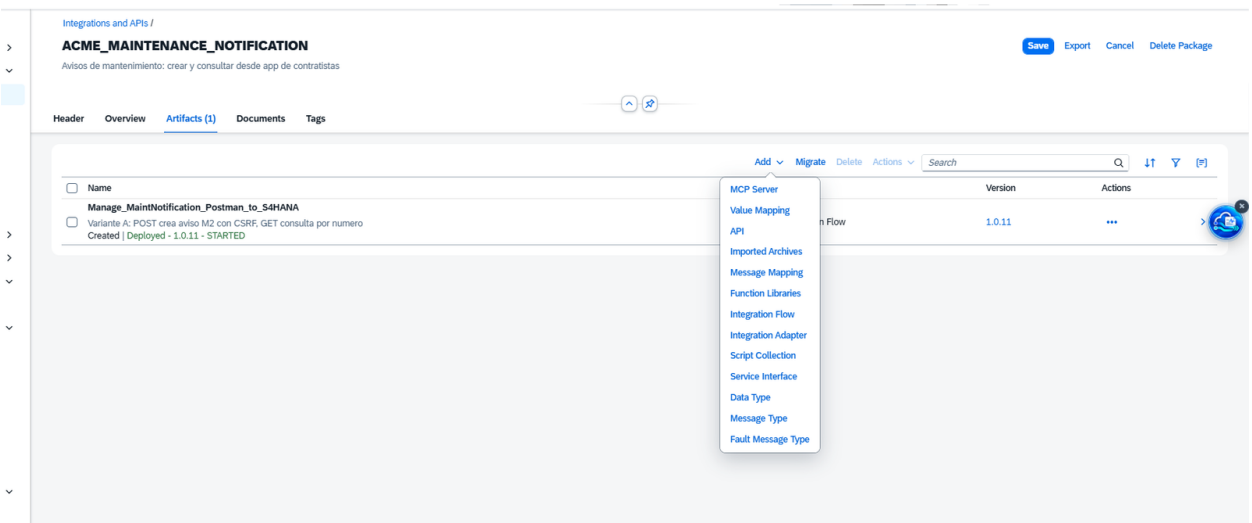
MCP cycle used by every client: initialize → notifications/initialized → tools/list → tools/call

2. Prerequisites

#	Requirement	How to verify
1	SAP Integration Suite tenant with **Integration Cell** active (trial works)	Settings > Runtimes > Integration Cell = Active
2	API Management (classic) active, with an API Proxy in front of your backend API	Configure > APIs lists the proxy; a GET with API key returns 200
3	Developer Hub with Admin Center access	Shell menu shows Developer Hub; Admin Center > Manage > Content opens
4	Role collections: PI_Integration_Developer, PI_Administrator, AuthGroup.API.Admin, AuthGroup.API.ApplicationDeveloper, Connectivity and Destination Administrator	BTP Cockpit > Security > Users
5	A working backend chain (here: iFlow → Cloud Connector → S/4HANA)	Call the existing API once before you start
6	An OpenAPI 3.0 document of the API with good description texts for each operation and field	Validate it with any OpenAPI linter
7	An MCP-capable client: Claude Desktop, or Bruno/Postman for raw JSON-RPC	—



Settings > Runtimes: Integration Cell must be Active, otherwise the MCP Server artifact is not offered



Integration package > Edit > Artifacts > Add: the MCP Server option appears when Integration Cell is active

> Tip: the MCP Server is generated from the OpenAPI document. Everything the agent will ever read about your Tools (names, descriptions, schemas, examples) comes from that file. Invest time there first.

3. Step 1 — Prepare the API Proxy in API Management

The existing iFlow receives the notification number as an HTTP **header** (notificationNumber). An MCP Server generated from OpenAPI calls GET /v1/{notificationNumber} with the number in the **path**. The proxy must translate path → header and must not forward the path suffix to Cloud Integration.

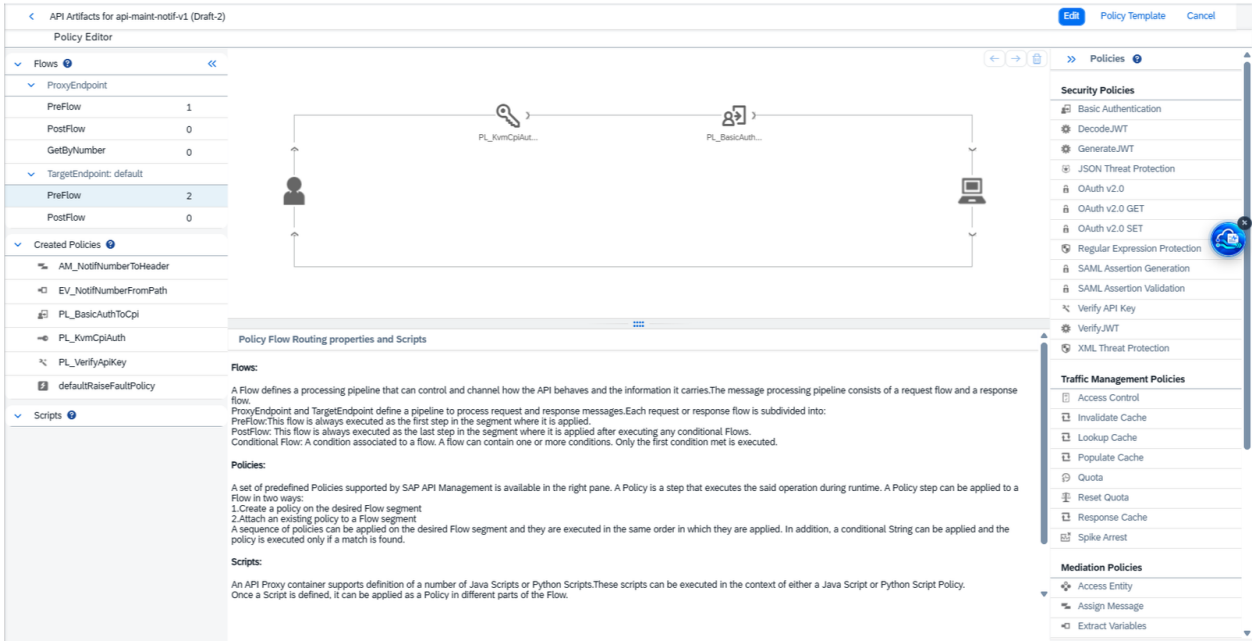
3.1 Create a conditional flow for the GET by path

Do **not** use the Resources tab to add GET /{notificationNumber}: the editor generates a read-only conditional flow whose condition contains the literal braces and never matches. Instead:

- Policies > **Edit** (if you skip Edit the editor is read-only and policies you add are silently not attached).
- ProxyEndpoint > **+** > conditional flow GetByNumber with condition (proxy.pathsuffix MatchesPath "/*") and (request.verb = "GET").
- In GetByNumber > Incoming Request add two policies, in this order:

```
<!-- EV_NotifNumberFromPath (Extract Variables) -->
<ExtractVariables async="false" continueOnError="false" enabled="true" xmlns="http://www.sap.com/apimgmt »
">
  <Source>request</Source>
  <URIPath><Pattern ignoreCase="true">/{notificationNumber}</Pattern></URIPath>
  <VariablePrefix>path</VariablePrefix>
  <IgnoreUnresolvedVariables>true</IgnoreUnresolvedVariables>
</ExtractVariables>


<!-- AM_NotifNumberToHeader (Assign Message) -->
<AssignMessage async="false" continueOnError="false" enabled="true" xmlns="http://www.sap.com/apimgmt">
  <Set><Headers><Header name="notificationNumber">{path.notificationNumber}</Header></Headers></Set>
  <IgnoreUnresolvedVariables>false</IgnoreUnresolvedVariables>
  <AssignTo createNew="false" type="request">request</AssignTo>
</AssignMessage>
```

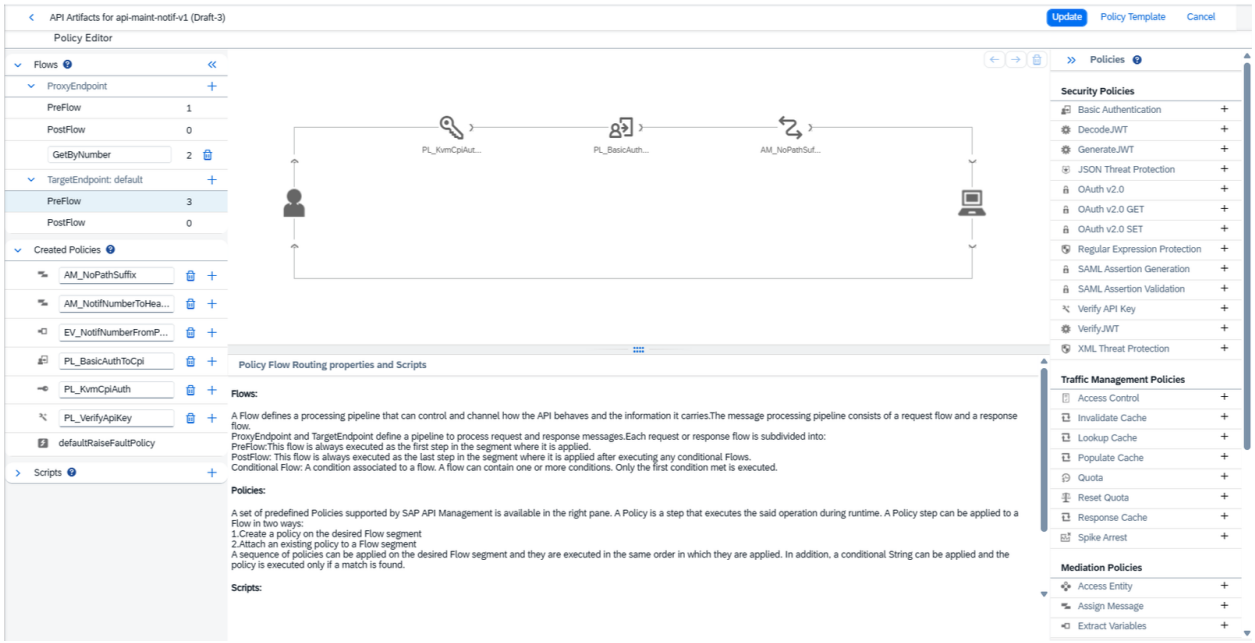


What failure looks like: the GetByNumber flow exists but shows 0 policies because Edit was not pressed before attaching them

3.2 Stop the path suffix from reaching the backend

target.copy.pathsuffix is a **target request** variable: setting it in the ProxyEndpoint has no effect and Cloud Integration answers a Tomcat 404 because /10000793 is appended to the iFlow address. Add this policy in **TargetEndpoint > PreFlow > Incoming Request**, after the Basic Auth policy that reads the Key Value Map:

```
<!-- AM_NoPathSuffix (Assign Message, TargetEndpoint PreFlow) -->
<AssignMessage async="false" continueOnError="false" enabled="true" xmlns="http://www.sap.com/apimgmt">
  <AssignVariable><Name>target.copy.pathsuffix</Name><Value>false</Value></AssignVariable>
  <IgnoreUnresolvedVariables>true</IgnoreUnresolvedVariables>
  <AssignTo createNew="false" type="request">request</AssignTo>
</AssignMessage>
```



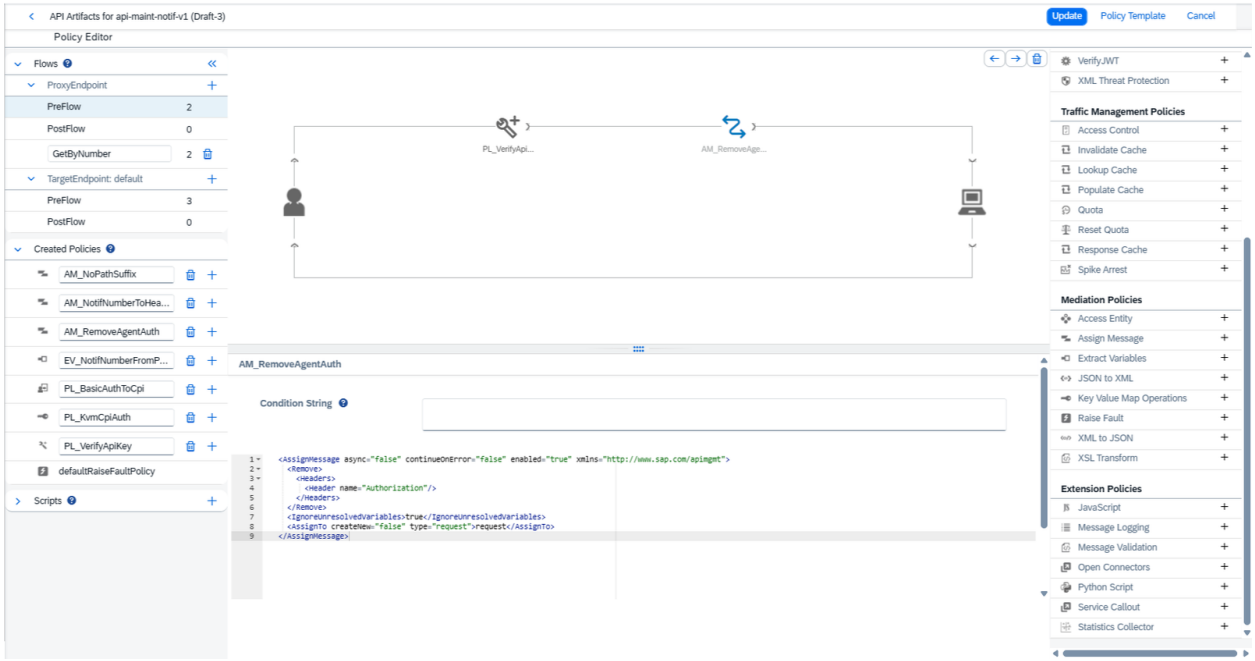
Final TargetEndpoint PreFlow: KVM lookup → Basic Auth to Cloud Integration → AM_NoPathSuffix

3.3 Remove the agent's Authorization header (do it now, you will need it later)

The MCP Server forwards the agent's Authorization: Bearer <token> to the backend. Cloud Integration recognises that XSUAA token as valid but without the ESBMessaging.send role and answers **403**. The Basic Auth policy does not overwrite the header, so drop it in the **ProxyEndpoint > PreFlow > Incoming Request**, after Verify API Key:

```
<!-- AM_RemoveAgentAuth (Assign Message, ProxyEndpoint PreFlow) -->
<AssignMessage async="false" continueOnError="false" enabled="true" xmlns="http://www.sap.com/apimgmt">
  <Remove><Headers><Header name="Authorization"/></Headers></Remove>
  <IgnoreUnresolvedVariables>true</IgnoreUnresolvedVariables>
  <AssignTo createNew="false" type="request">request</AssignTo>
</AssignMessage>
```

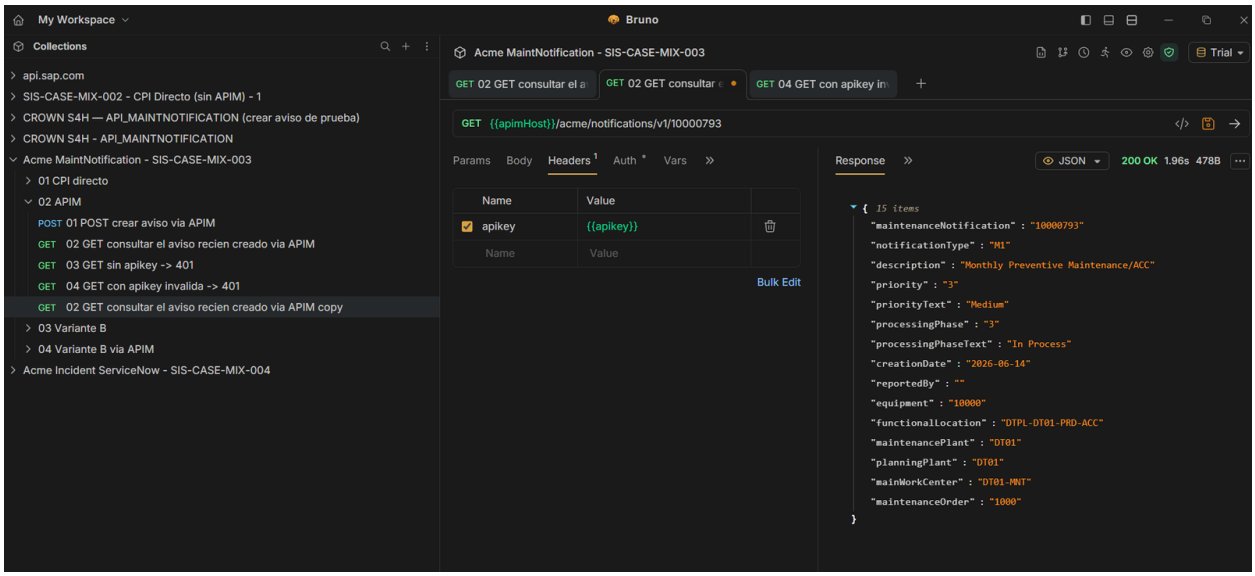
Why the ProxyEndpoint and not the TargetEndpoint: the policy editor always appends a new policy at the **end** of the selected flow and does not let you reorder. In the TargetEndpoint it would run after Basic Auth and delete the backend credentials too. In the ProxyEndpoint it always runs before anything in the TargetEndpoint.



ProxyEndpoint PreFlow after the change: Verify API Key → AM_RemoveAgentAuth

3.4 Save, deploy, verify

Update > Save > Deploy. Then call the proxy with the API key only: GET /acme/notifications/v1/10000793 must return 200 with the notification JSON. Also re-run the original header-based calls: nothing should have changed for existing consumers.



Verification from Bruno: GET by path with API key only → 200, maintenanceOrder 1000

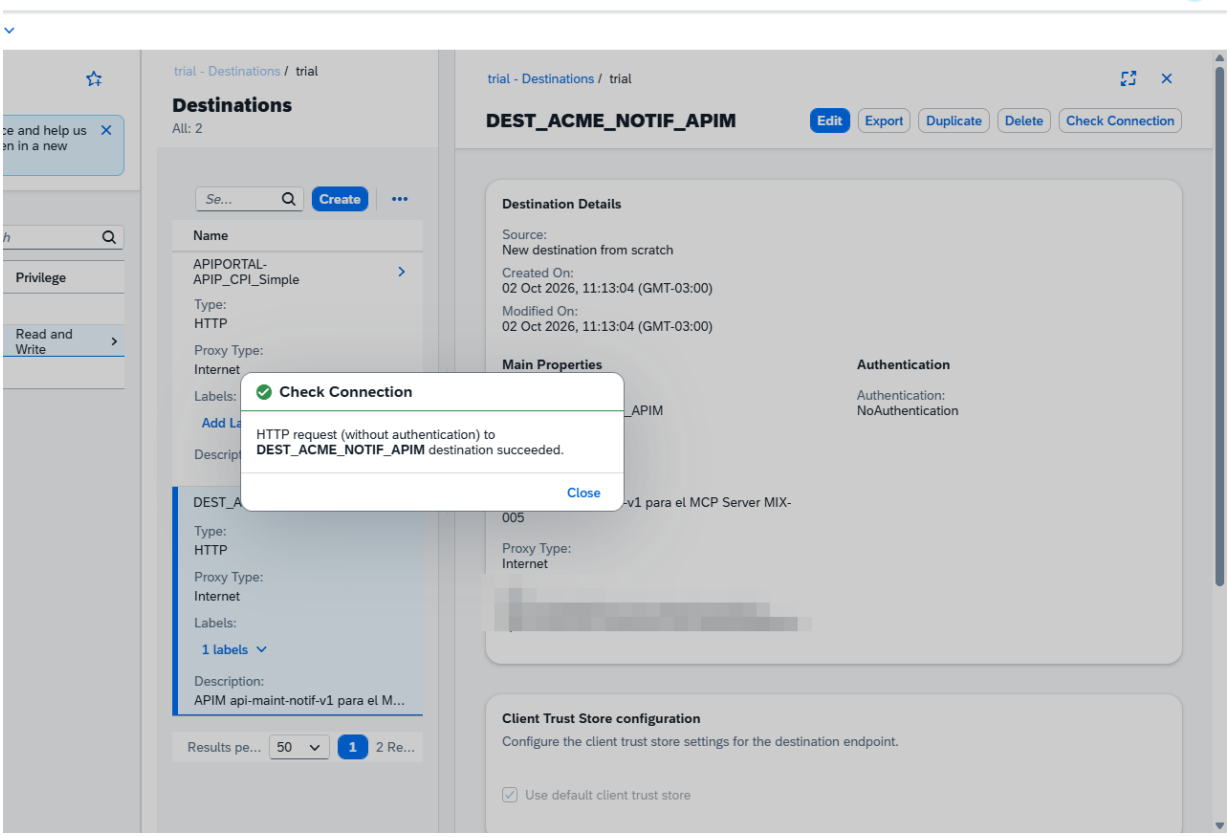
Do not add <DisplayName> to any policy XML: SAP API Management rejects it ("Invalid content starting with DisplayName"). Also save API resources with an empty description; the rich-text editor triggers "Unable to update API - internal error".

4. Step 2 — Create the BTP Destination

BTP Cockpit > subaccount > Connectivity > Destinations > Create.

Field	Value
Name	DEST_ACME_NOTIF_APIM
Type / Proxy Type	HTTP / Internet
URL	API Management host, e.g. https://<tenant>-trial.integrationsuitetrial-apim.<region>.hana.ondemand.com:443/<tenant>
Authentication	NoAuthentication (the proxy is protected by API key)
Additional property	URL.headers.apikey = API key of the application subscribed to the API product
Label	IntegrationCell.Include=true (required so the destination is offered in the MCP Server wizard)

The MCP Server never sees the API key: the Destination service injects the header. Press **Check Connection**: "HTTP request (without authentication) succeeded" is the expected result.

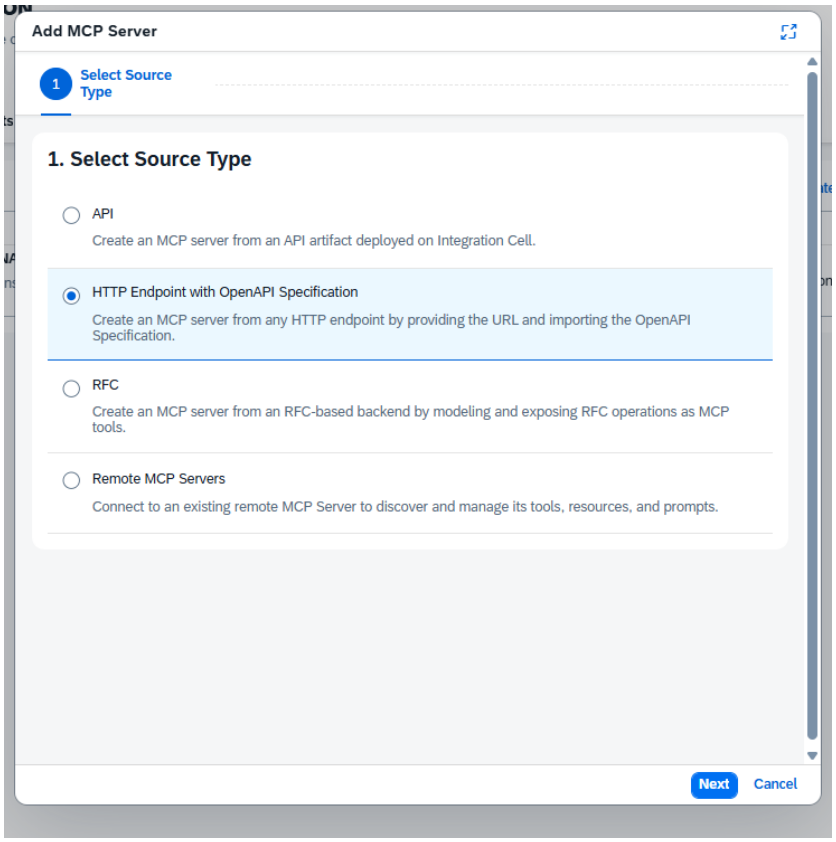


Destination with the APIM host, NoAuthentication and the label; Check Connection succeeded

5. Step 3 — Create the MCP Server

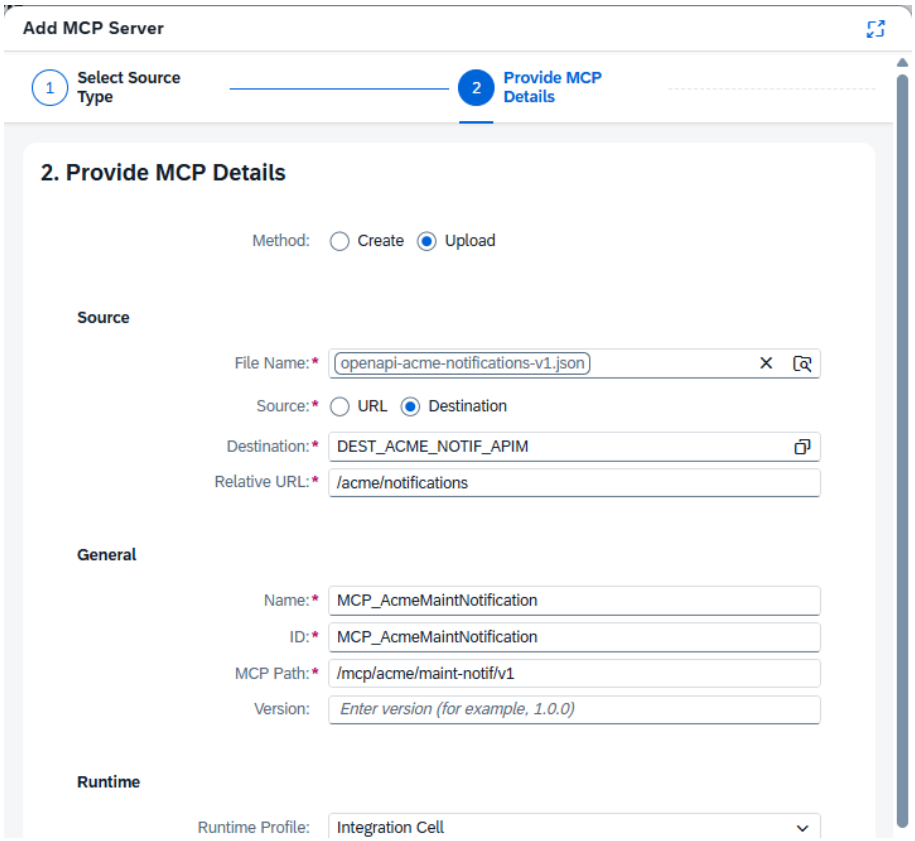
Design > Integrations and APIs > your package > Edit > Artifacts > Add > **MCP Server**.

5.1 Wizard

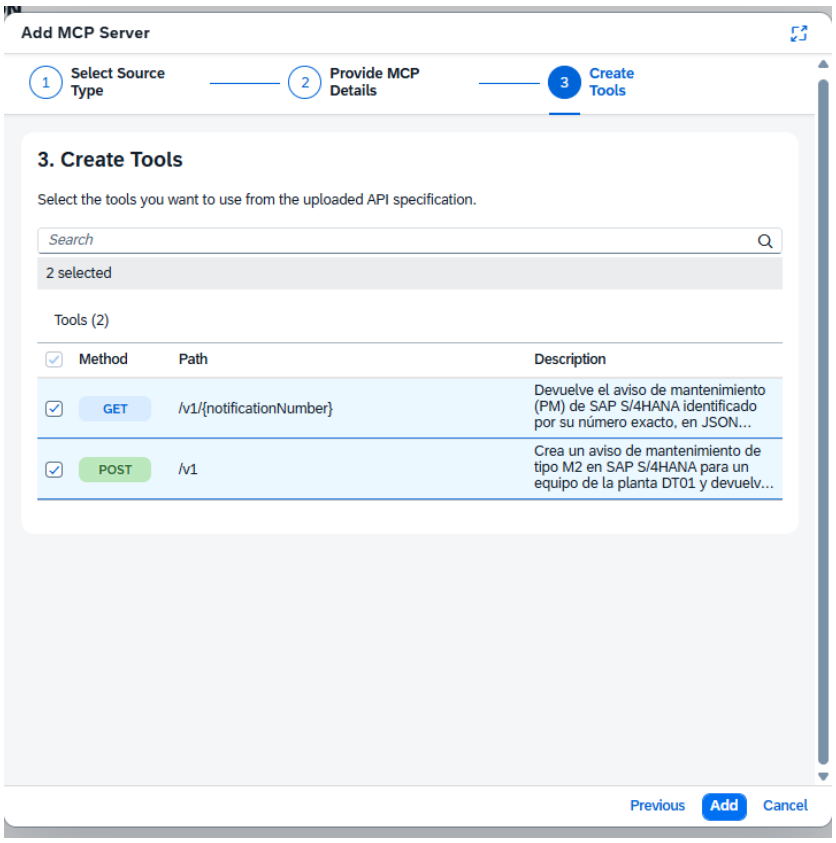


Step 1: Source Type = HTTP Endpoint with OpenAPI Specification (other options: API artifact, RFC, Remote MCP Server)

Field	Value used
Method	Upload (the OpenAPI JSON file)
Source	Destination DEST_ACME_NOTIF_APIM
Relative URL	/acme/notifications (the /v1/... part comes from the OpenAPI paths)
Name / ID	MCP_AcmeMaintNotification (the wizard copies the name into the ID)
MCP Path	/mcp/acme/maint-notif/v1
Runtime Profile	Integration Cell
Virtual Host	the default Integration Cell host



Step 2: upload the OpenAPI document, point the source to the Destination, set the MCP Path and runtime

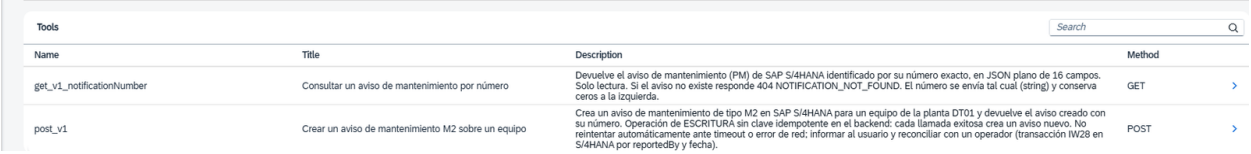


Step 3: pick the operations that become Tools (GET /v1/{notificationNumber}, POST /v1)

5.2 Rename the Tools and write the descriptions for the model

The gateway **ignores operationId** and names Tools after method and path: get_v1_notificationNumber, post_v1. Rename them under MCP Configuration > Tools and rewrite the descriptions. These texts are the user interface of

your API for the LLM.



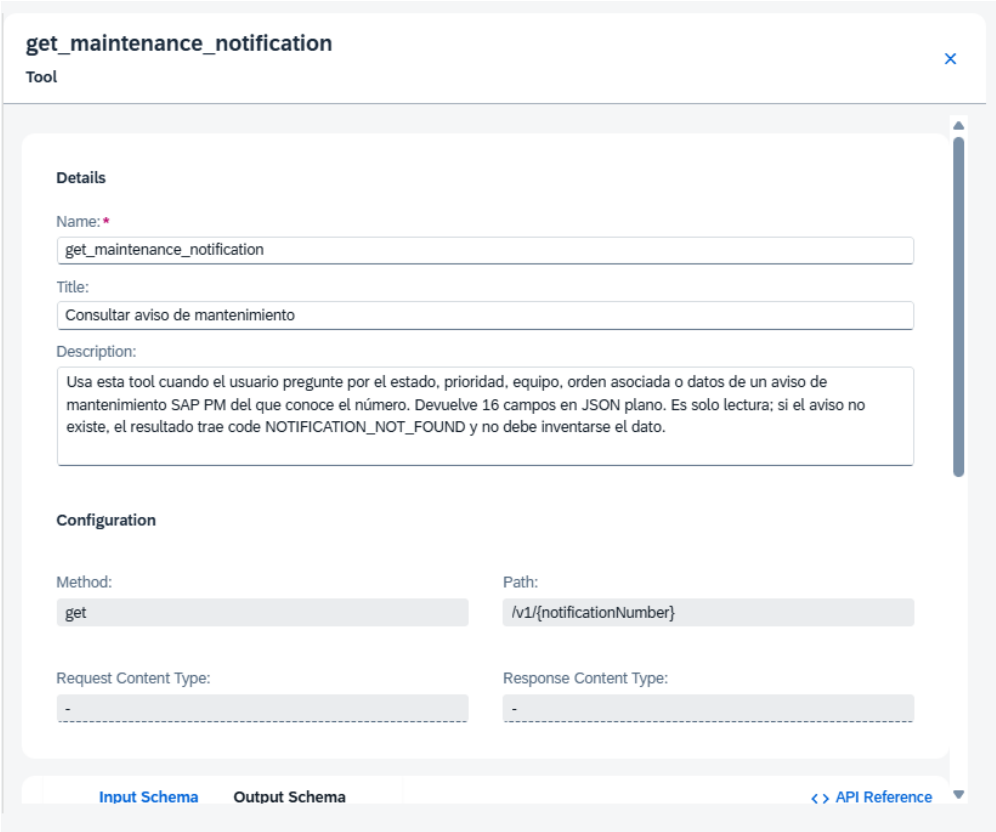
Tools

Search

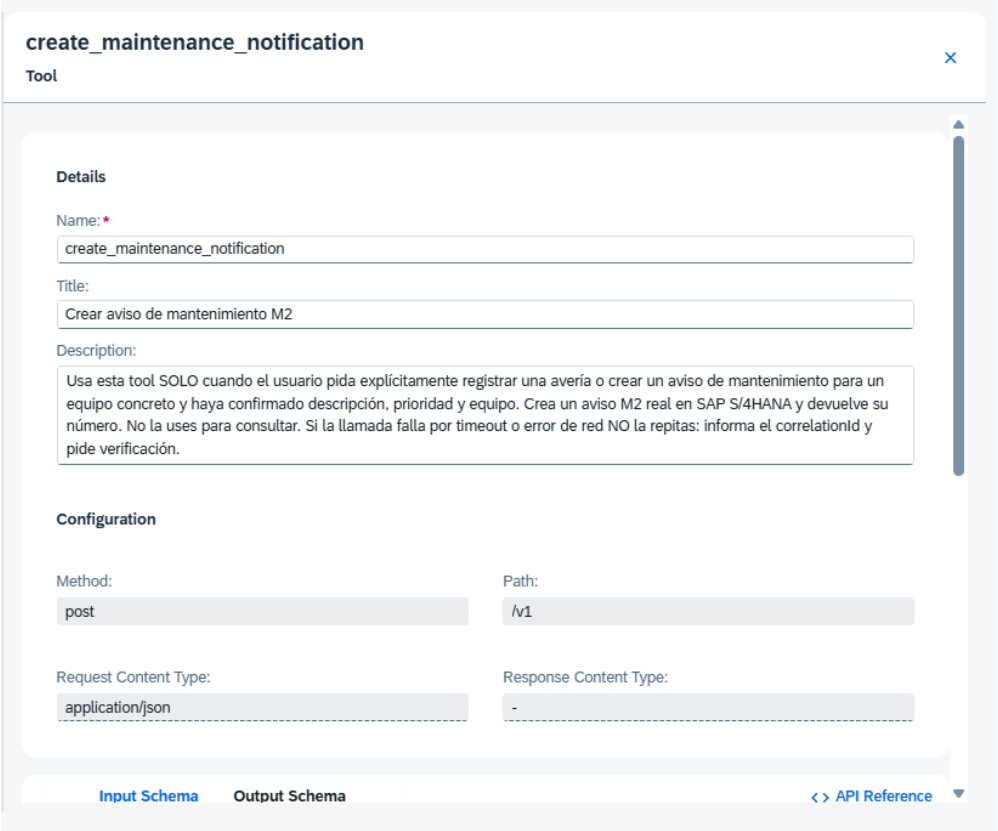
Name	Title	Description	Method
get_v1_notificationNumber	Consultar un aviso de mantenimiento por número	Devuelve el aviso de mantenimiento (PM) de SAP S/4HANA identificado por su número exacto, en JSON plano de 16 campos. Solo lectura. Si el aviso no existe responde 404 NOTIFICATION_NOT_FOUND. El número se envía tal cual (string) y conserva ceros a la izquierda.	GET
post_v1	Crear un aviso de mantenimiento M2 sobre un equipo	Crea un aviso de mantenimiento de tipo M2 en SAP S/4HANA para un equipo de la planta DT01 y devuelve el aviso creado con su número. Operación de ESCRITURA sin clave idempotente en el backend: cada llamada exitosa crea un aviso nuevo. No reintentar automáticamente ante timeout o error de red; informar al usuario y reconciliar con un operador (transacción IW28 en S/4HANA por reportedBy y fecha).	POST

Tools as generated: get_v1_notificationNumber and post_v1

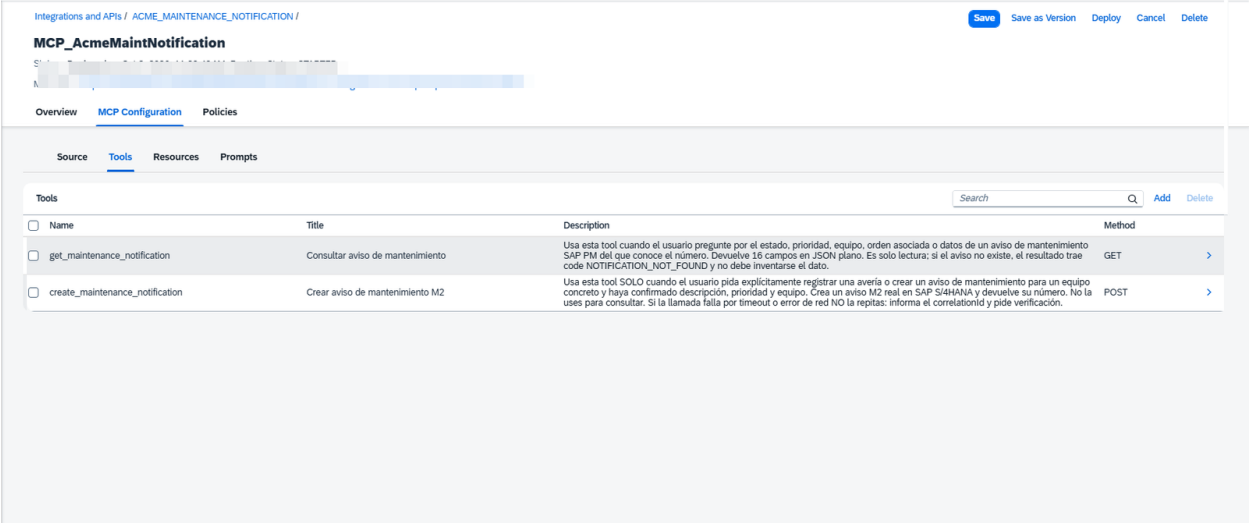
Tool	Description that worked
get_maintenance_notification	"Use this tool when the user asks for the status, priority, equipment, linked order or data of an SAP PM maintenance notification whose number is known. Returns 16 flat JSON fields. Read-only; if the notification does not exist the result carries code NOTIFICATION_NOT_FOUND and the data must not be invented."
create_maintenance_notification	"Use this tool ONLY when the user explicitly asks to register a malfunction or create a maintenance notification for a specific equipment and has confirmed description, priority and equipment. Creates a real M2 notification in SAP S/4HANA and returns its number. Do not use it to query. If the call fails with a timeout or network error do NOT retry: report the correlationId and ask for verification."



get_maintenance_notification after renaming, with the agent-oriented description



create_maintenance_notification: the "ONLY when ... has confirmed" sentence is the main safeguard for writes

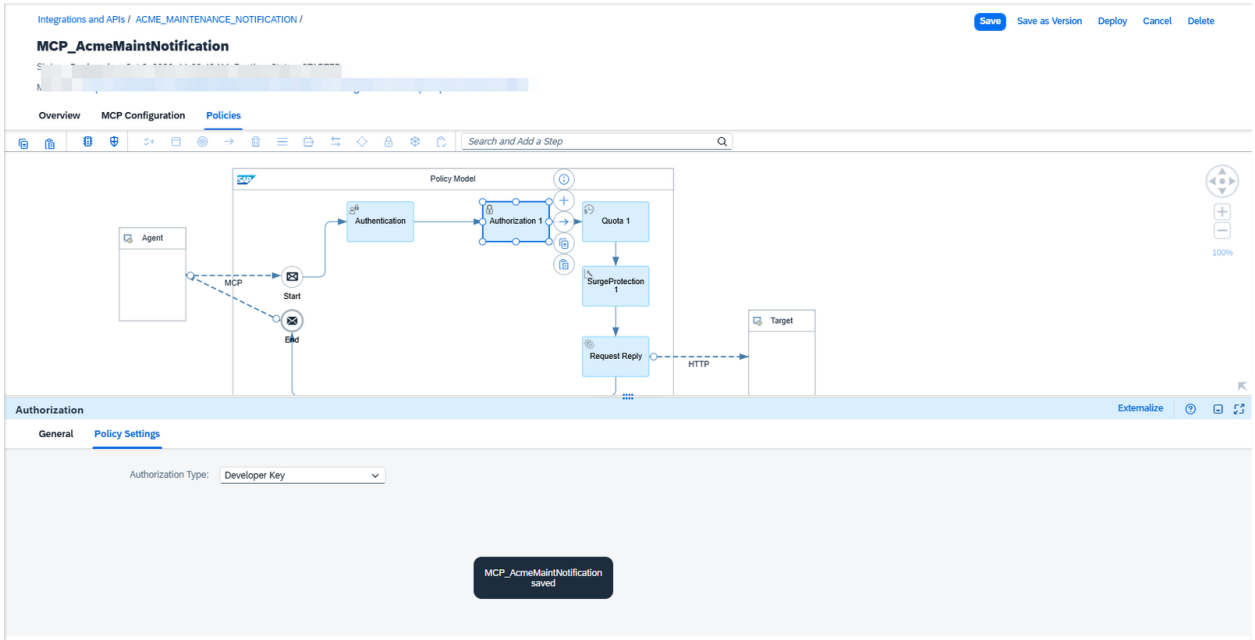


Final Tools list

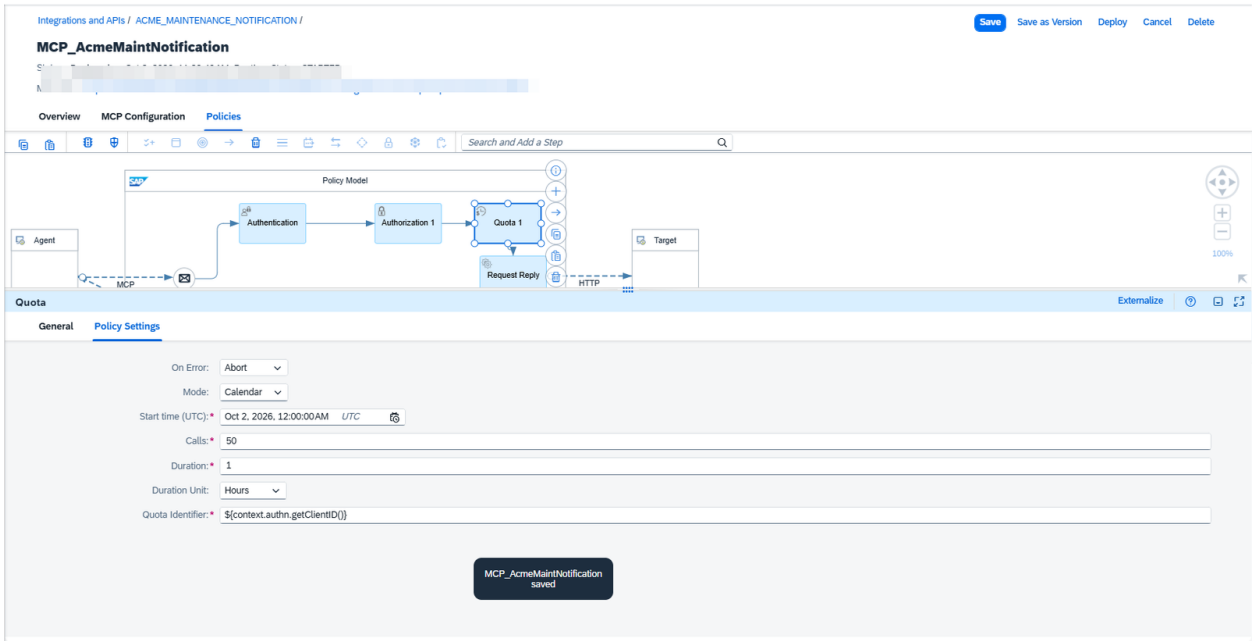
5.3 Policies

The Policies tab shows the default model Authentication → Authorization → Request Reply. Configure:

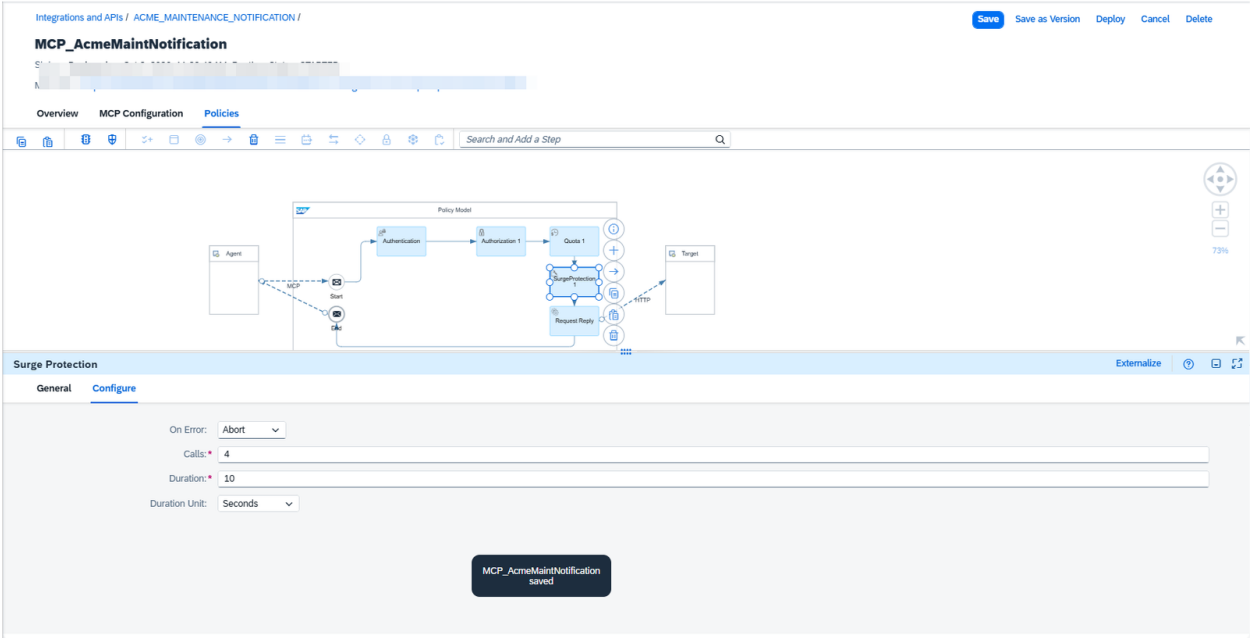
Policy	Setting
Authentication	Tenant XSUAA, Client Certificate + OAuth (the agent presents a Bearer token issued to a Developer Hub subscription)
Authorization	**Developer Key**: only clients subscribed to the product may call
Quota	50 calls per hour, Calendar mode, identifier ${context.authn.getClientID()}
Surge Protection	4 calls per 10 seconds → HTTP 429



Authorization type Developer Key



Quota: 50 calls / 1 hour, Calendar mode, keyed by client ID

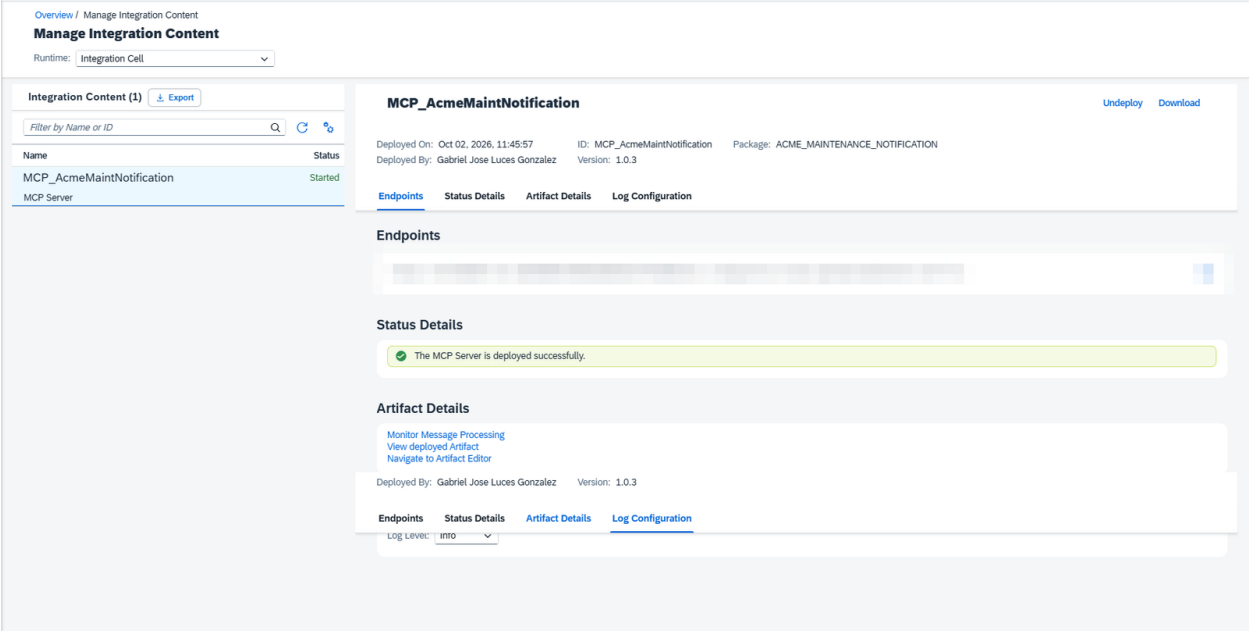


Surge Protection: 4 calls / 10 seconds

> Agents can loop. Quota and Surge Protection are the only server-side limits on how much damage a confused agent can do with a write Tool.

5.4 Save as Version, Deploy, confirm

Save as Version (semantic version), Deploy, then Monitor > Manage Integration Content with Runtime = Integration Cell. The artifact must show **Started** and the endpoint https://<integration-cell-host>/mcp/acme/maint-notif/v1.

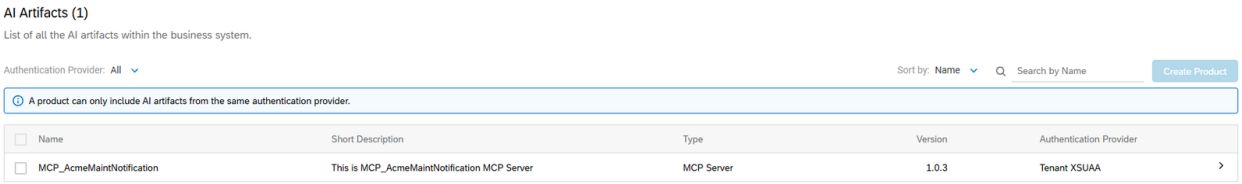


Manage Integration Content: MCP Server Started with its endpoint

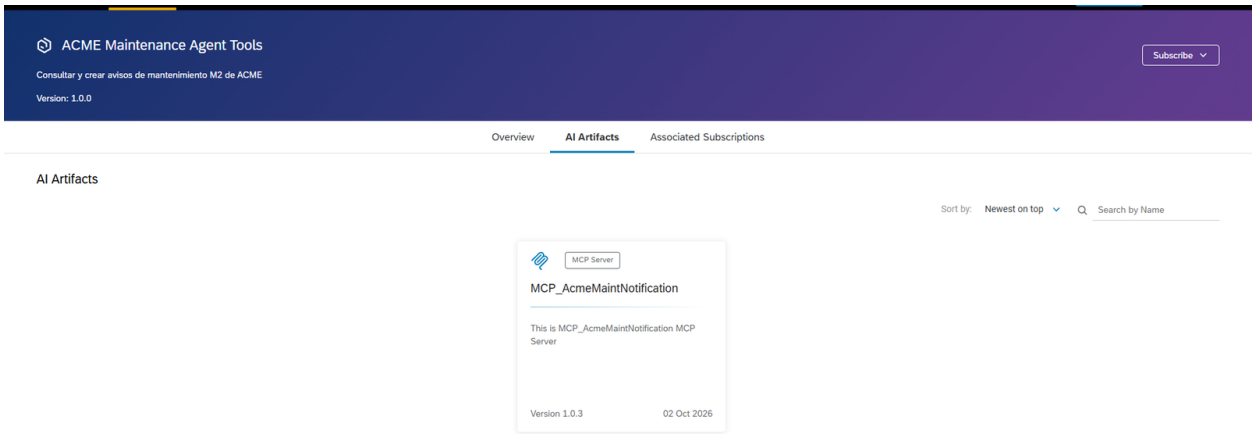
6. Step 4 — Publish in Developer Hub and issue agent credentials

6.1 Product

Admin Center > Manage > Content > Business Systems > your Integration Suite system > tab **AI Artifacts** (there is no "MCP Servers" tab). Select the MCP Server > Create Product > publish.

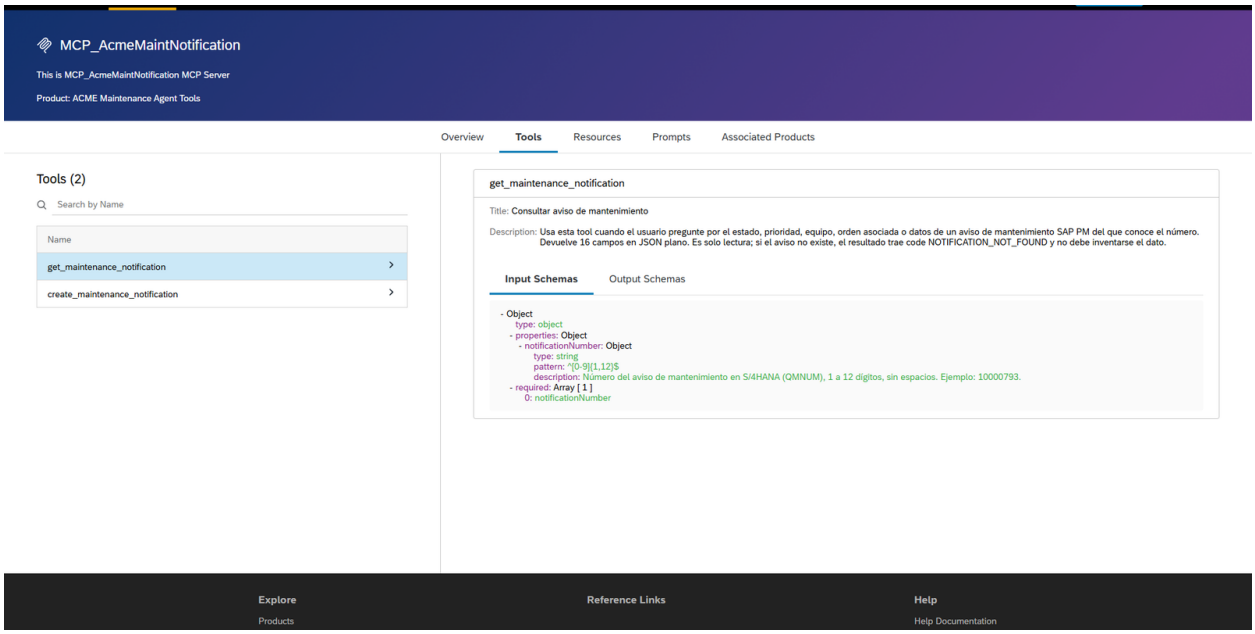


AI Artifacts tab listing the MCP Server with Authentication Provider = Tenant XSUAA



Published product with the MCP Server as its AI artifact

The catalogue renders the Tools with their input schemas, which is exactly what the agent will receive in tools/list:



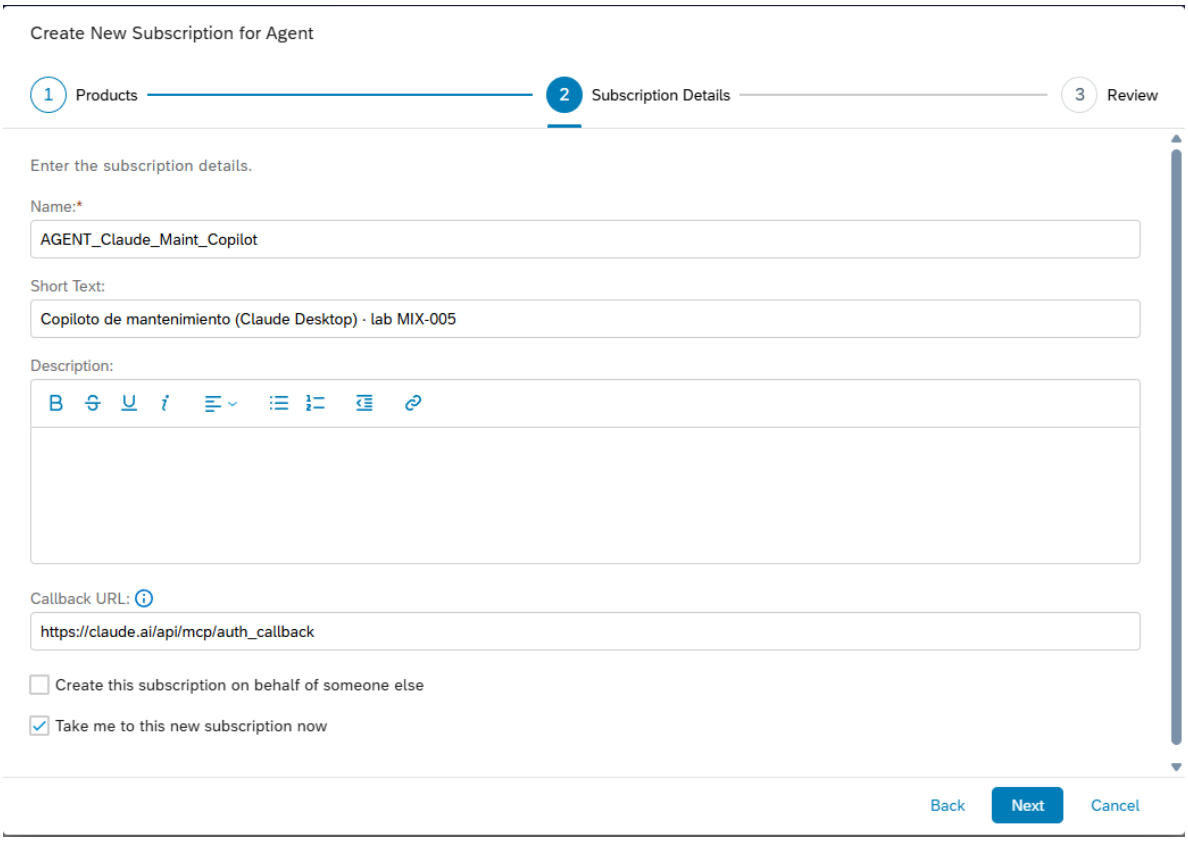
Catalogue view of get_maintenance_notification: notificationNumber pattern ^[0-9]{1,12}$

Catalogue view of create_maintenance_notification: the body is wrapped in a requestBody property

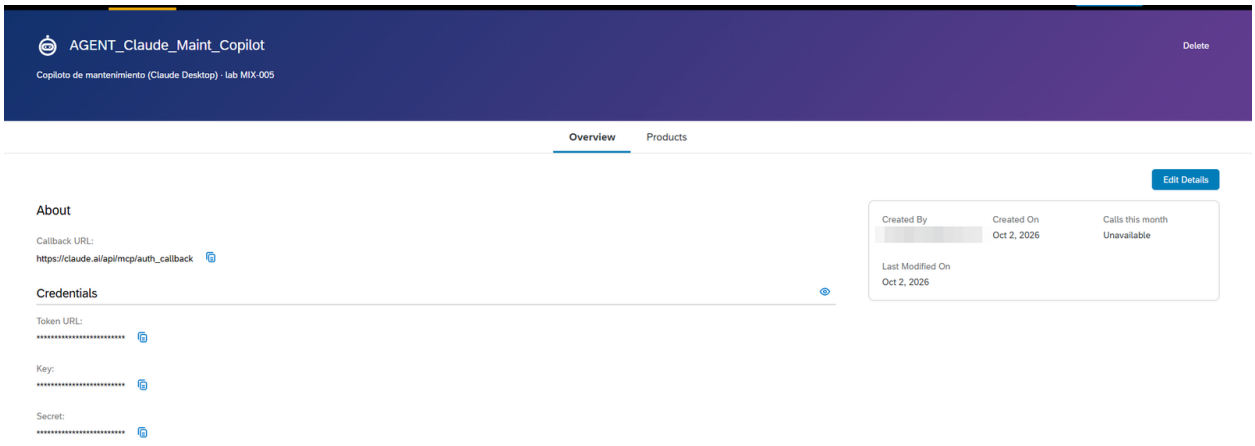
POST-based Tools wrap the body in **requestBody**. An LLM reads the schema and gets it right; hand-written test collections do not, until you notice.

6.2 Subscription for Agent

My Workspace > Create > **New Subscription for Agent** > select the product > name it (AGENT_Claude_Maint_Copilot) > Callback URL https://claude.ai/api/mcp/auth_callback > Save. After about a minute (refresh), the Overview shows **Token URL, Key and Secret**.



Subscription wizard: name, short text and callback URL



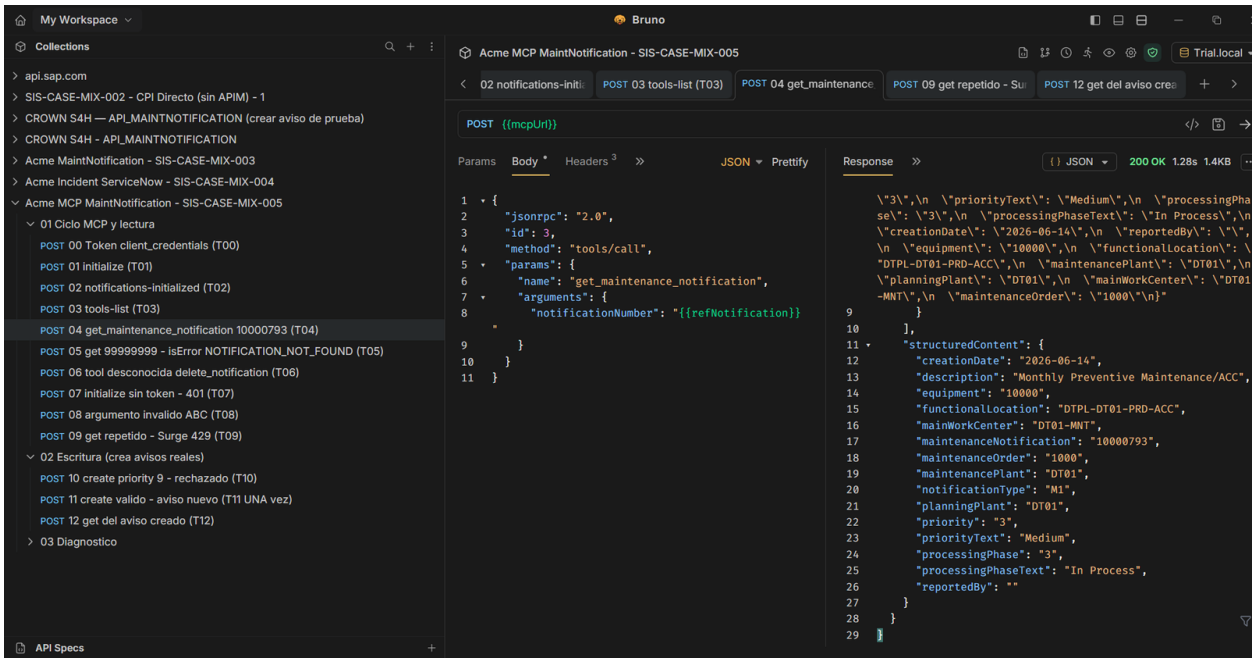
Credentials issued to the agent subscription (masked)

Copy them to a credential manager only. The token obtained with client_credentials has scope uaa.resource and an audience that includes the Cloud Integration runtime, which is why Step 3.3 matters.

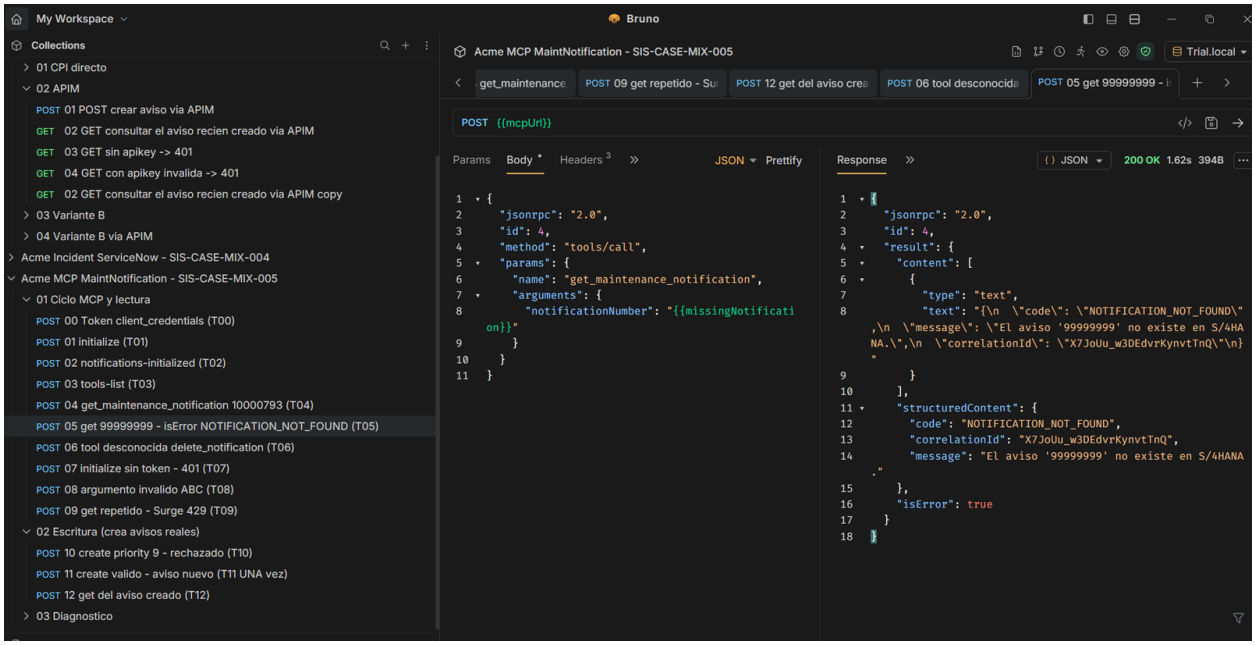
7. Step 5 — Test the raw MCP cycle (Bruno or Postman)

Every request is a POST to the MCP endpoint with headers Content-Type: application/json, Accept: application/json, text/event-stream, MCP-Protocol-Version: 2025-06-18, Authorization: Bearer <token> and, after initialize, Mcp-Session-Id if the server returned one.

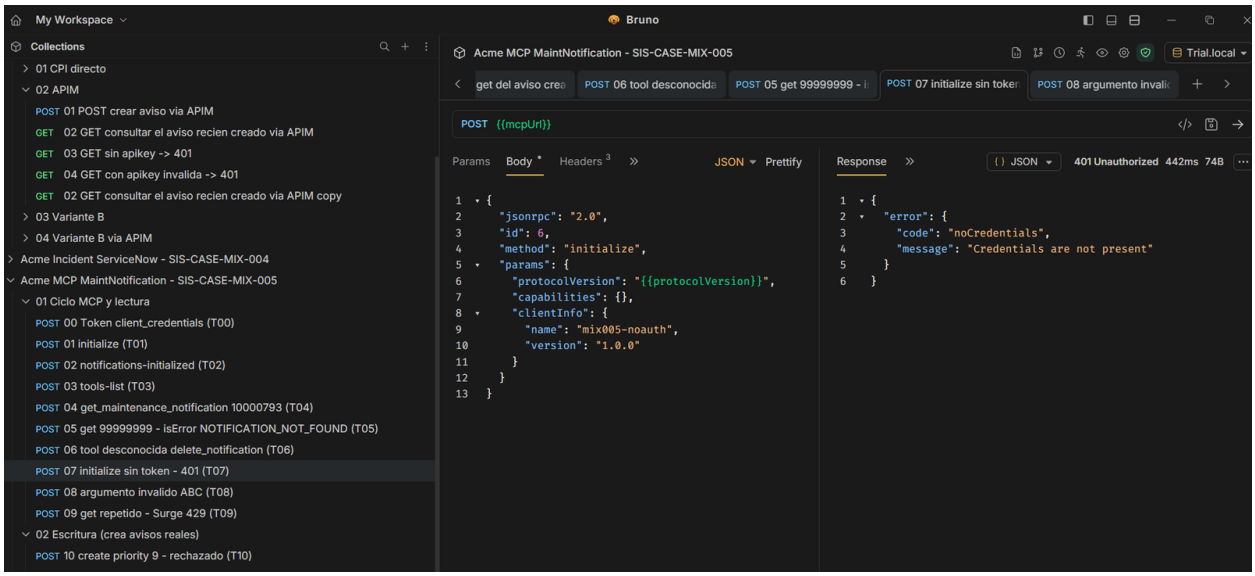
#	Request	Expected	Observed
T00	POST Token URL, grant_type=client_credentials, Basic Key:Secret	200, access_token	OK (expires in 3599 s)
T01	initialize (protocolVersion, capabilities, clientInfo)	200, serverInfo, capabilities.tools	OK
T02	notifications/initialized	202/204	202
T03	tools/list	exactly the two Tools with inputSchema	OK
T04	tools/call get_maintenance_notification {notificationNumber: "10000793"}	200, structuredContent with the notification	OK, maintenanceOrder 1000
T05	get with 99999999	200 with isError: true, code NOTIFICATION_NOT_FOUND	OK
T06	tools/call delete_notification (not exposed)	JSON-RPC error -32602 unknown tool	OK
T07	initialize without Bearer	401	401 noCredentials
T08	get with "ABC"	isError (pattern violation)	OK, rejected by the gateway
T09	5 × get within 10 s	5th call 429	pending
T10	create with priority 9	isError (enum violation), no notification created	OK
T11	create valid, run once	200, new notification number	OK, 10000965
T12	get the new number	same notification	OK



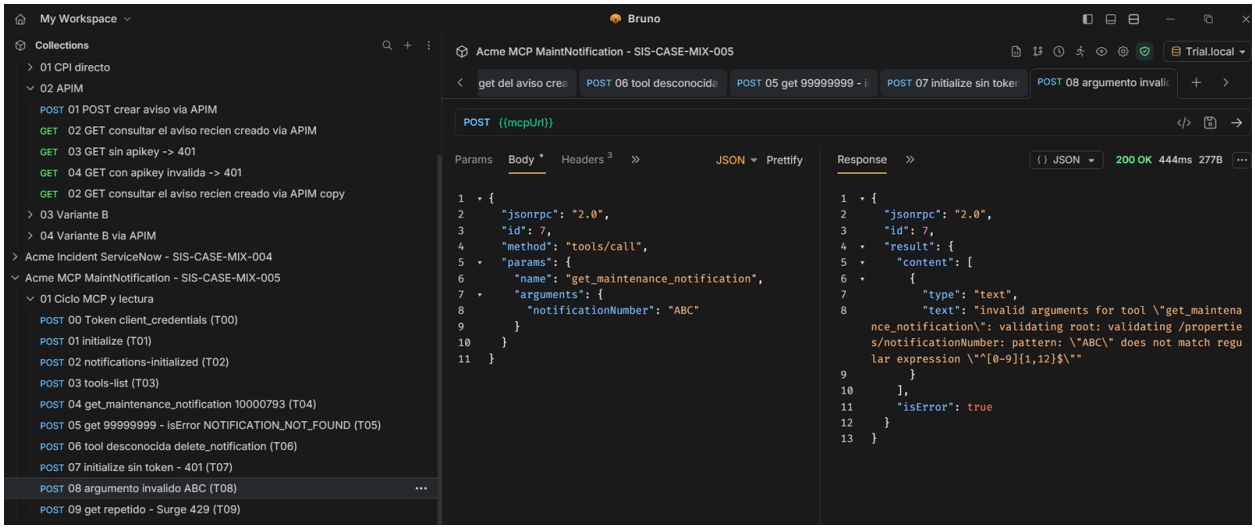
T04: tools/call returns content[].text and structuredContent with the notification fields



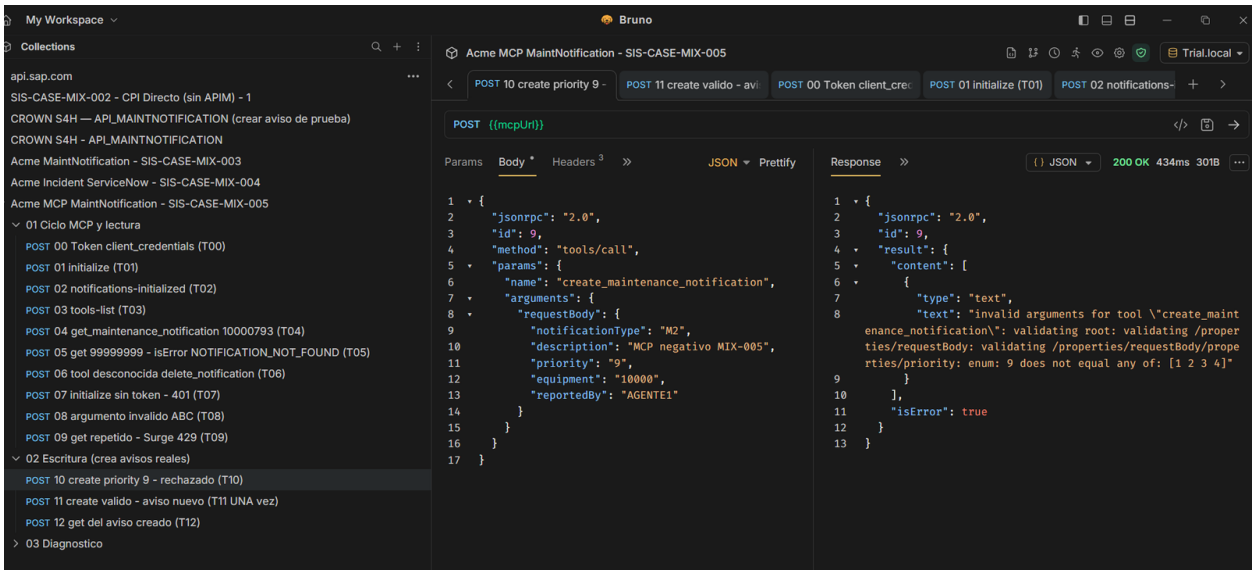
T05: business 404 arrives as a Tool error (isError true, code NOTIFICATION_NOT_FOUND, correlationId), not as a protocol error



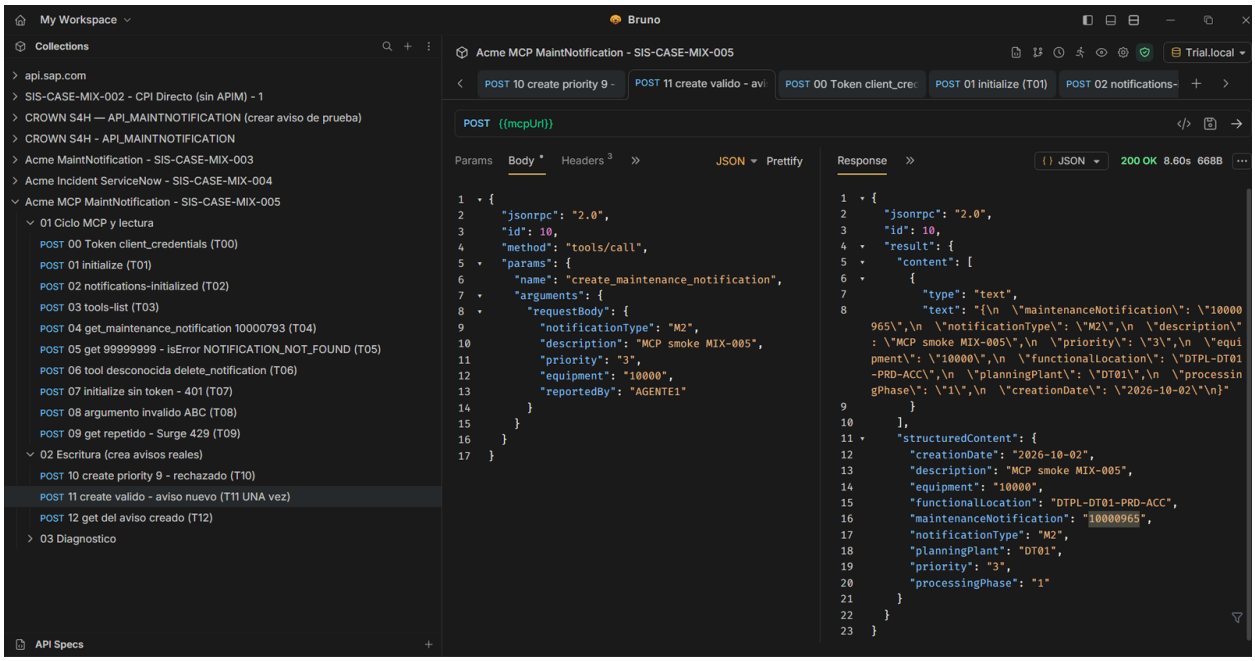
T07: no Bearer → 401 noCredentials from the Authentication policy



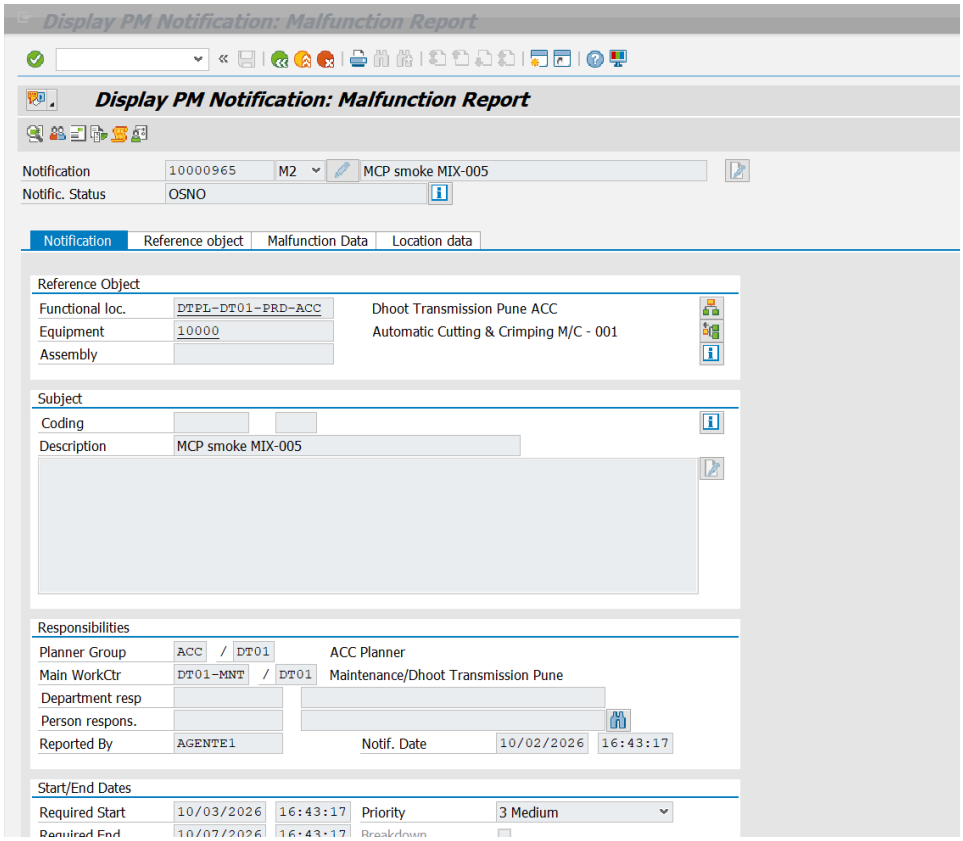
T08: "ABC" violates the pattern of the input schema; the gateway rejects before calling the backend



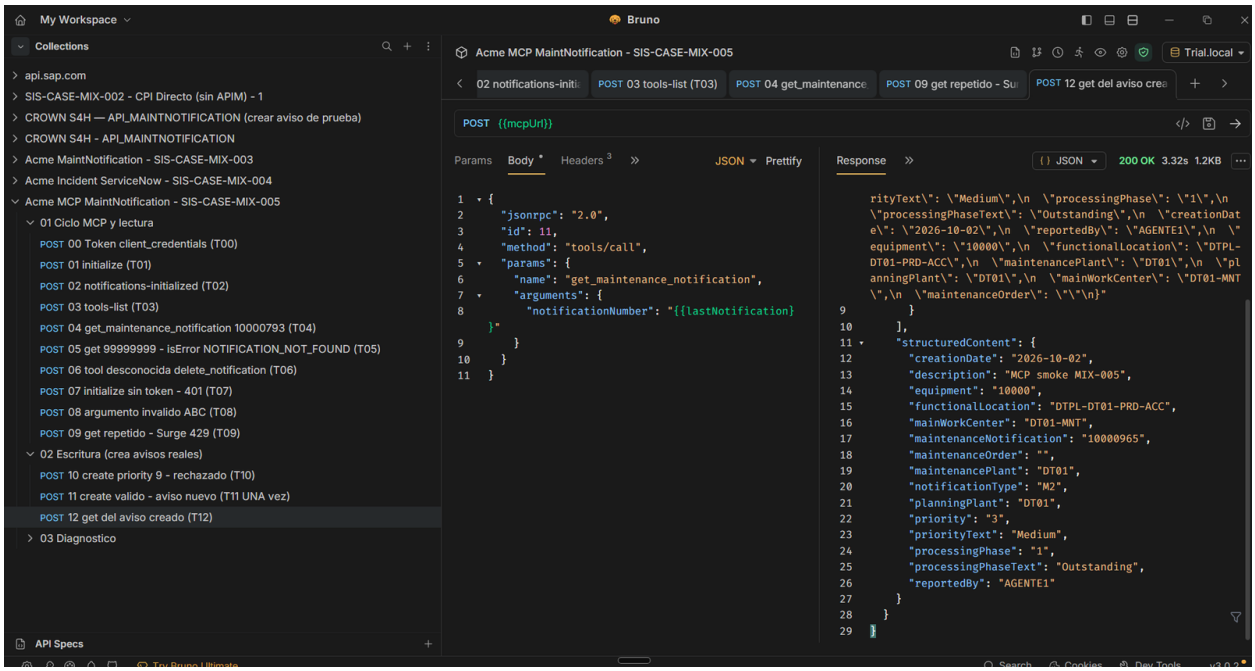
T10: priority 9 violates the enum; rejected without reaching S/4HANA



T11: create with arguments.requestBody → notification 10000965 created



IW23 in S/4HANA: notification 10000965, type M2, status OSNO, equipment 10000, work centre DT01-MNT



T12: reading the new notification through the MCP Server

Finally re-run the regression of the original API consumers (API key clients): a POST and a GET through API Management must still return 201 and 200.

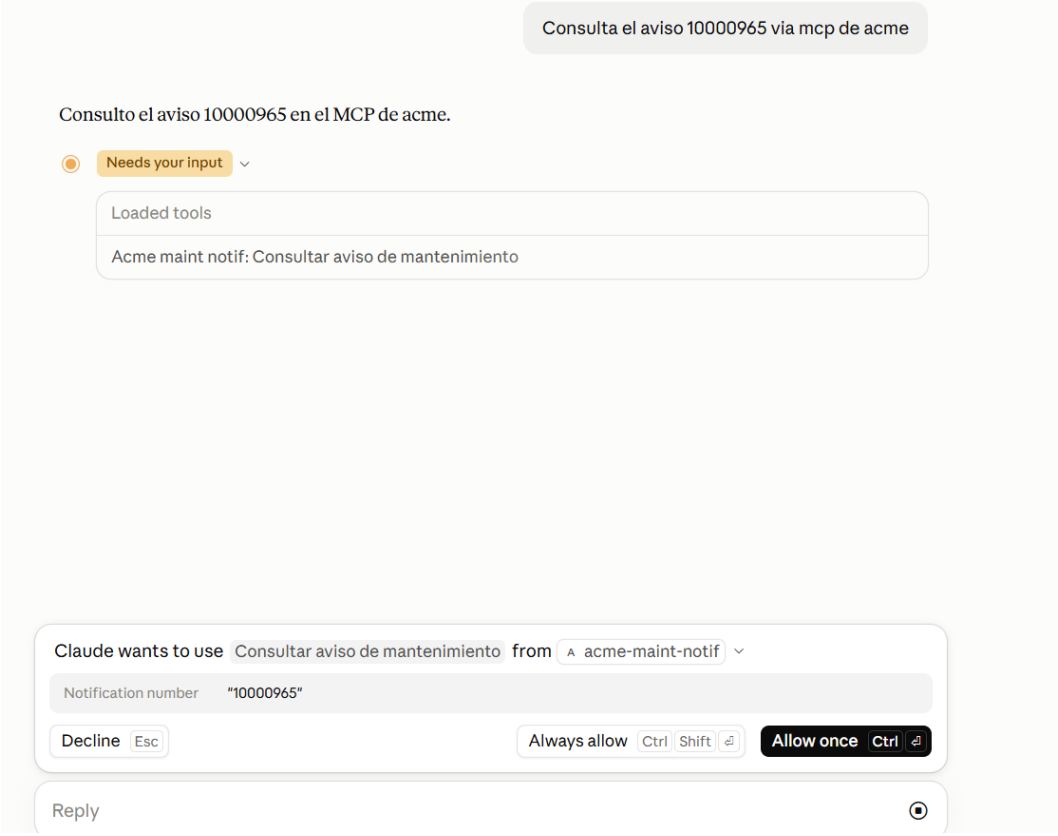
8. Step 6 — Connect Claude Desktop

The subscription issues an OAuth client for client_credentials. Claude's remote connector expects authorization_code, so the simplest reliable path is a small local **stdio** → **Streamable HTTP bridge** that Claude Desktop launches as an MCP server. It obtains the token, refreshes it 60 s before expiry (or on 401), keeps the Mcp-Session-Id (re-initialises on 404) and forwards every JSON-RPC message. Node 18+ is enough; no dependencies.

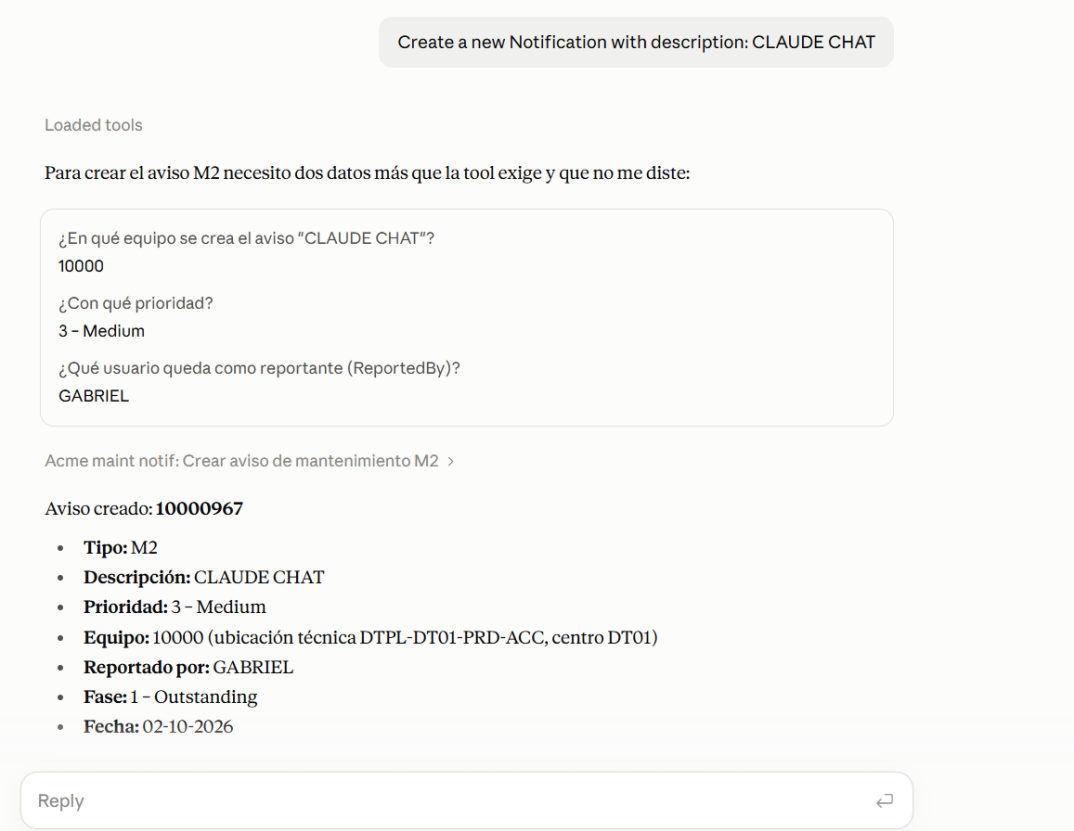
Register it in claude_desktop_config.json with Claude Desktop **fully closed** (system tray > Quit); the app rewrites the file on exit and your entry disappears otherwise:

```
"acme-maint-notif": {
  "command": "C:\\path\\to\\node.exe",
  "args": [
    "C:\\path\\to\\acme-mcp-bridge.mjs",
    "C:\\path\\to\\credentials.local"
  ]
}
```

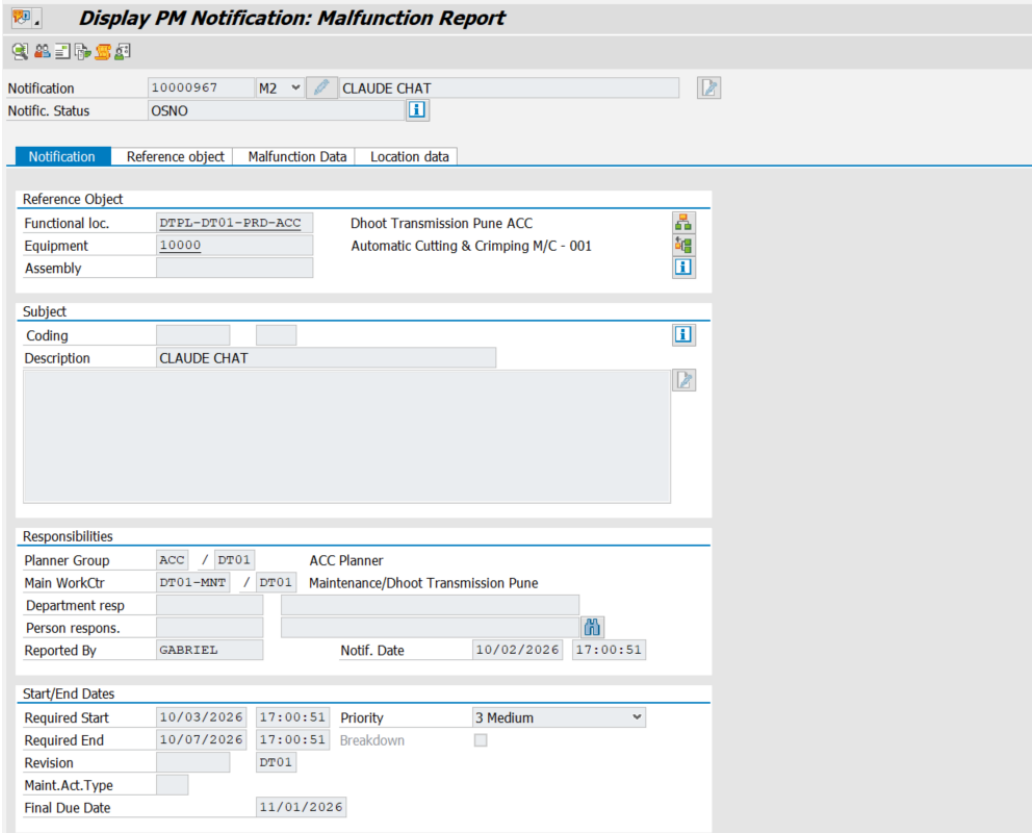
Restart Claude Desktop; the tools menu lists acme-maint-notif with two tools.



Claude asks permission before calling the read Tool with notificationNumber 10000965



Claude asks for equipment, priority and reporter before calling the write Tool, then reports notification 10000967



IW23: notification 10000967 "CLAUDE CHAT" created by the agent, reporter GABRIEL

> The confirmation before writing comes from the Tool description plus Claude Desktop's per-call permission prompt. It is a behavioural safeguard, not a technical one: document it as a risk and keep Quota and Surge Protection on.

9. Troubleshooting: the five real errors

Symptom	Cause	Fix
"Unable to update API - internal error" when saving a Resource	Description typed in the rich-text editor	Save the resource with an empty description
"Unable to parse policy ...; Invalid content starting with DisplayName"	SAP APIM schema rejects <DisplayName>	Remove it from every policy XML
GET /v1/{n} → 404 HTML (Tomcat) from Cloud Integration; conditional flow shows 0 policies	Auto-generated flow with literal braces never matches; target.copy.pathsuffix set in ProxyEndpoint has no effect; editor in read-only mode	Own flow with MatchesPath "/*"; AM_NoPathSuffix in TargetEndpoint PreFlow; press Edit before attaching
tools/call create → isError: true "required: missing properties: [requestBody]"	POST Tools wrap the body in requestBody	Send arguments.requestBody{...} (agents do this automatically)
tools/call get → isError: true with HTML "403 Forbidden" while initialize/tools/list work	The agent's Bearer is forwarded to Cloud Integration; the token is valid but lacks the messaging role	AM_RemoveAgentAuth in ProxyEndpoint PreFlow after Verify API Key

10. Lessons

- Reuse the API you already have: the MCP Server is a layer, not a rewrite. Zero changes to the iFlow in this build.
- Tool name + description are the agent's UI. Write them for the model: when to use, what comes back, what never to do (retry a write).
- Validation in the input schema (pattern, enum, required) stops bad calls at the gateway before they touch SAP.
- Check which headers reach the backend. The agent's token must stop at the gate.

- Quota, Surge Protection and Authorization by subscription are mandatory for write Tools.
- Business errors should travel as isError: true with a code and a correlationId; the agent can explain them to the user instead of hallucinating.
- Rotate the credentials you used while testing and never commit them: keep them in one local file outside version control.

Appendix A — Minimal OpenAPI conventions that produced good Tools

- One description per operation written as an instruction to the agent (see 5.2).
- pattern on identifiers (^[0-9]{1,12}$), enum on controlled values (priority 1–4), required lists, maxLength on free text (40 chars for the notification short text).
- Flat JSON responses: nested structures become harder for the model to summarise.
- Keep the operation set small (2 Tools here). Every Tool is a decision the model has to make.

Appendix B — Bruno environment used for the tests

```
mcpUrl           https://<integration-cell-host>/mcp/acme/maint-notif/v1
tokenUrl         https://<subaccount>.authentication.<region>.hana.ondemand.com/oauth/token
protocolVersion  2025-06-18
clientId / clientSecret   from the Developer Hub subscription (secret vars)
refNotification  10000793      missingNotification  99999999
```